

Image Geolocation Estimation App

USER MANUAL

Application Version 2.4

Proof of Concept (POC)

Updated October 2026

Important POC Notice

Proof-of-Concept Status

Image Geolocation Estimation App v2.4 remains a proof of concept. It is intended for local testing, demonstrations, coursework, and controlled experimentation. It is not a production-hardened multi-user service.

The current build has been tested on Windows. It has not been formally tested or validated on Linux or macOS. Although the application architecture is largely cross-platform, compatibility on those operating systems should not be assumed until testing is completed.

- The current documented installation procedure is Windows-oriented and uses PowerShell.
- AI similarity scores are ranking aids, not proof that a candidate image is the same location as the target.
- Mapillary imagery coverage varies by location; absence of a match does not prove that the target is not in the search area.
- The current local architecture is single-user and does not provide authentication or per-user isolation.
- The Mapillary token is exposed to the browser when MapillaryJS is used; do not deploy this POC unchanged to untrusted users.

Current v2.4 Known-Issue Summary

Read Before Testing

Four current POC limitations are especially important: (1) repeated identical Mapillary searches can return different candidate counts; (2) the date filters currently trigger a Mapillary timezone error and should be left blank; (3) after a Large Area Search, the user must still click Load combined results before reviewing or ranking the deduplicated set; and (4) very long Windows installation paths can cause SQLite disk I/O errors.

About This Manual

This manual explains how to install, configure, start, and operate Image Geolocation Estimation App v2.4. It describes every user-facing field, button, checkbox, and pull-down menu in the current interface, including Standard Search, Large Area Search, saved runs, shortlist management, DINOv2 ranking, Pano/Flat AI filtering, and the current known limitations.

Table of Contents

1. What the Application Does
2. System Requirements
3. Files in the ZIP Package
4. Installing from the ZIP File
5. Adding the Mapillary API Key
6. Installing the Optional AI Components
7. Starting and Stopping the Application
8. Recommended First-Run Workflow

- 9. Search Mode and Geographic Search Controls
- 10. Standard Search
- 11. Large Area Search
- 12. Saved Runs and Shortlist
- 13. AI Visual Similarity Ranking
- 14. Target Photograph Controls
- 15. Candidate Viewer Controls
- 16. Current Image Metadata
- 17. Recommended AI Workflows
- 18. Hardware Acceleration
- 19. Files, Database, Cache, and Backups
- 20. CSV Exports
- 21. Troubleshooting
- 22. Security and Privacy Notes
- 23. Known Limitations and POC Boundaries
- 24. Quick Reference

1. What the Application Does

Image Geolocation Estimation App helps a human investigator estimate the likely location of a target photograph by searching street-level imagery inside a user-defined geographic area and ranking candidate Mapillary images with local AI visual similarity.

Core Workflow

Target photograph → geographic search → Mapillary candidate discovery → optional AI ranking → shortlist → deeper review → human geolocation judgment.

The application uses Mapillary as its imagery provider and DINOv2 (facebook/dinov2-small) as the local vision model. Candidate embeddings are compared with the target using cosine similarity. Fast Multi-Crop is intended for broad screening, while Thorough Multi-Crop performs a more expensive second-stage comparison.

Version 2.4 supports both Standard Search and checkpointed Large Area Search. Large Area Search divides a large circular search region into overlapping 1,500-meter cells, processes them sequentially, deduplicates Mapillary image IDs, and stores progress in SQLite.

2. System Requirements

Requirement	Current v2.4 Requirement / Recommendation
Operating system	Windows 10/11 recommended and tested. Linux and macOS remain untested.
Python	A current Python 3 installation. Python 3.13.5 has been used successfully with the POC.
Browser	A modern browser such as Microsoft Edge, Google Chrome, or Firefox.
Internet	Required for Mapillary imagery/API access. Also required for the initial DINOv2 model download unless already cached.
Mapillary credential	A valid Mapillary client access token.
Disk space	Allow several GB for Python/PyTorch packages, model cache, uploaded targets, SQLite data, and optional Mapillary thumbnail cache.
RAM	More memory is beneficial for very large candidate sets and AI workloads.
GPU	Optional. CPU is supported. Accelerator paths include AMD ROCm, NVIDIA CUDA, Apple MPS, and optional DirectML.

Windows Path-Length Recommendation

Use a Short Installation Path

On Windows, install the application in a short path such as D:\IGE_v2_4 or D:\Image_Geolocation_Estimation_App_v2_4. A deeply nested path can cause SQLite to fail with `sqlite3.OperationalError: disk I/O error` even when the drive itself is healthy.

If the app fails during database initialization and the application directory is deeply nested, move the entire folder to a shorter path and run `serve.py` again before troubleshooting SQLite itself.

3. Files in the ZIP Package

File	Purpose
serve.py	Local web server, Mapillary API access, SQLite persistence, target uploads, Large Area Search orchestration, and AI job API.
ai_matcher.py	DINOv2 model loading, hardware detection, thumbnail retrieval/cache, crop generation, and similarity scoring.
index.html	Main browser interface.
app.js	Client-side search, reviewer, Large Area Search, shortlist, filtering, sorting, and AI logic.
style.css	User-interface styling.
config.json	Optional local Mapillary token configuration. A clean distributed package should contain a blank token.
config.example.json	Example Mapillary-token configuration.
requirements-ai.txt	Optional AI package requirements.
install_ai.ps1	Windows PowerShell helper for installing AI dependencies.
check_ai_hardware.py	Reports the compute backends visible to the application.
START_HERE.txt	Condensed startup instructions.
README.md	Technical notes and version information.
UPGRADE_FROM_v2_3.txt	Upgrade guidance for a v2.3 installation.

4. Installing from the ZIP File

4.1 Extract the ZIP

1. Download the v2.4 ZIP package.
2. In File Explorer, right-click the ZIP and choose Extract All.
3. Choose a short, writable path, preferably near the root of a drive. Example: D:\IGE_v2_4.
4. Do not run the application directly from inside the ZIP archive.

4.2 Confirm Python

Open PowerShell in the extracted application folder and run:

```
python --version
```

If Windows reports that Python is not installed or not found, install Python 3 and ensure the Python executable is available from PowerShell.

4.3 Optional Virtual Environment

Recommended but Optional

A Python virtual environment helps keep the application's packages separate from other Python projects.

```
python -m venv .venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

4.4 Upgrading from v2.3

When upgrading an existing v2.3 installation, the update-only package is preferable because it is designed not to overwrite the existing Mapillary configuration, SQLite database, target uploads, or AI thumbnail cache.

- Stop `serve.py` before copying update files.
- Back up the existing application folder first.
- Preserve `mapillary_runs.sqlite3`, `config.json`, `ai_cache`, and `target_uploads`.
- The v2.4 database schema upgrades automatically when the server starts.

5. Adding the Mapillary API Key

A valid Mapillary client access token is required before Mapillary imagery can be discovered or displayed.

5.1 Environment Variable

In the same PowerShell session that will start the application:

```
$env:MAPILLARY_ACCESS_TOKEN="MLY|YOUR_TOKEN_HERE"
```

Then start the application with:

```
python .\serve.py
```

5.2 config.json

```
{
  "mapillary_access_token": "MLY|YOUR_TOKEN_HERE"
}
```

Security Warning

`config.json` stores the token as plaintext. Do not distribute a configured `config.json` or commit it to a public repository.

5.3 Configuration Priority

If both methods are present, `MAPILLARY_ACCESS_TOKEN` takes priority over `config.json`.

5.4 Confirming Configuration

After `serve.py` starts, the upper-right Mapillary badge should report that the token is configured. If it reports that the token is not configured, discovery will not run.

6. Installing the Optional AI Components

Mapillary discovery and manual review can run without DINOv2. AI ranking requires the optional packages.

6.1 Standard Installation

```
.\install_ai.ps1
```

If needed, install manually:

```
python -m pip install -r .\requirements-ai.txt
```

The requirements include:

- PyTorch
- Torchvision
- Transformers
- Pillow
- NumPy
- Safetensors

6.2 First AI Run

The DINOv2 model facebook/dinov2-small is downloaded automatically by Hugging Face Transformers the first time it is needed and reused from the local Hugging Face cache thereafter.

6.3 GPU-Specific PyTorch

GPU acceleration depends on the PyTorch build installed in the active Python environment. For AMD ROCm systems, preserve a known-working ROCm-enabled PyTorch installation and install the remaining application dependencies around it.

Run the hardware diagnostic:

```
python .\check_ai_hardware.py
```

7. Starting and Stopping the Application

7.1 Start

```
python .\serve.py
```

The browser normally opens automatically. If it does not, open:

```
http://localhost:8000/
```

7.2 Optional Startup Commands

Command	Purpose
<code>python .\serve.py --port 8001</code>	Use a different local port if 8000 is occupied.
<code>python .\serve.py --no-browser</code>	Start the server without opening a browser automatically.
<code>python .\serve.py --host 127.0.0.1</code>	Bind explicitly to the local machine; this is the normal local-only behavior.

7.3 Stop

Return to the PowerShell window and press:

```
Ctrl+C
```

8. Recommended First-Run Workflow

1. Install the application in a short Windows path.
2. Configure the Mapillary access token.

3. Start serve.py and open the browser interface.
4. Upload the target photograph.
5. Choose Standard Search for a radius of 1,500 meters or less, or Large Area Search for a larger radius.
6. Enter the center latitude/longitude and search settings.
7. Run Mapillary discovery.
8. If using Large Area Search, wait for cells to finish, then click Load combined results.
9. Check the AI environment.
10. Choose Both, Panoramas only, or Flat images only for the AI input set.
11. Run Fast Multi-Crop on the broad candidate set.
12. Review top matches and add promising images to the shortlist.
13. Load the shortlist and run Thorough Multi-Crop.
14. Save the current run if AI scores and timing statistics should persist.

9. Search Mode and Geographic Search Controls

Control	Type	What It Does
Search mode	Pull-down	Selects Standard Search or Large Area Search.
Center latitude	Numeric field	Latitude of the search center. Required.
Center longitude	Numeric field	Longitude of the search center. Required.
Radius (meters)	Numeric field	Standard Search radius. Default 402.336 m; maximum 1,500 m.
Overall radius (meters)	Numeric field	Large Area Search radius. Valid range 1,501–50,000 m.
Image type	Pull-down	Discovery filter: All imagery, Panoramas only, or Flat images only.
Captured on/after	Date field	Optional capture-date lower bound. CURRENT v2.4 BUG: leave blank; see Known Limitations.
Captured on/before	Date field	Optional capture-date upper bound. CURRENT v2.4 BUG: leave blank; see Known Limitations.
Maximum API images	Numeric field	Safety cap on API images examined. In Large Area Search this is applied per 1,500 m cell.

Discovery Image Type

- All imagery — requests panorama and non-panorama imagery.
- Panoramas only — discovery is limited to panorama images.
- Flat images only — discovery is limited to non-panorama images.

The discovery Image type control is separate from the v2.4 AI image-type filter. Discovery determines what Mapillary imagery is collected; the AI image-type filter determines which images from the already-loaded candidate set are sent to DINOv2.

10. Standard Search

Standard Search is intended for circular searches with a radius of 1,500 meters or less.

Control	What It Does
Discover Mapillary images	Starts the Mapillary spatial query, filters returned imagery back to the requested circle, deduplicates image IDs, and initially orders candidates by distance from the center.
Discovery status	Reports API pages, unique API images examined, number of images inside the circle, errors, and other progress information.

Current Reproducibility Limitation

In v2.4, repeated identical Standard Searches can return different candidate counts and different image IDs. Treat the current Mapillary discovery set as a POC result rather than an exhaustive deterministic inventory.

The current implementation uses Mapillary bounding-box discovery and then applies exact circular filtering locally. The difference between API images examined and images inside the circle is expected because the API query is rectangular; large differences between repeated identical runs are a known issue under investigation.

11. Large Area Search

Large Area Search is designed for radii greater than 1,500 meters. It creates overlapping 1,500-meter sub-search cells, stores the cell plan in SQLite, and processes cells sequentially.

Control	What It Does
Large Area Search job name	Optional name for the checkpointed discovery job.
Start Large Area Search	Creates the overlapping cell grid, stores it, and starts sequential Mapillary discovery.
Saved Large Area Search jobs	Selects a previously created Large Area Search job.
Pause	Requests a safe pause; completed cells remain saved.
Cancel	Stops the active Large Area Search while preserving completed-cell data.
Resume	Continues pending cells without rerunning completed cells.
Retry failed cells	Retries only cells currently marked Failed.
Load combined results	Loads the job's globally deduplicated Mapillary image set into the normal reviewer and AI workflow.
Delete job	Deletes the Large Area Search job, its cell/checkpoint state, and its stored combined discovery results.

11.1 Cell Geometry

- Sub-search radius: 1,500 m
- Approximate center spacing: 2,450 m
- Approximate row spacing: 2,122 m
- Cells intentionally overlap to reduce coverage gaps.

11.2 Scrollable Cell List

Version 2.4 displays all generated cells in a fixed-height scrollable table. The table header remains visible while scrolling. The active running cell is brought into view when it starts, but the user can manually scroll to inspect other cells. Running and failed cells receive visual emphasis.

The cell table includes:

- Cell identifier
- Center coordinates
- Status
- Image count
- Attempt count
- API-cap status
- Error information

11.3 Job Status

Typical cell states include Pending, Running, Complete, and Failed.

A failed cell does not necessarily terminate the complete Large Area Search. The job can continue to other cells, and failed cells can be retried separately.

11.4 Load Combined Results

Required in v2.4

After useful Large Area Search results are available, click Load combined results to make the deduplicated image set the current reviewer candidate set. This step is still manual in v2.4.

Load combined results does not rerun the Mapillary search. It reads the stored deduplicated Large Area Search results and places them into the normal reviewer so they can be browsed, AI-ranked, shortlisted, saved, or exported.

11.5 Delete Job vs. Saved-Run Delete

Delete job removes the Large Area Search operation and its cell/checkpoint/discovery data. The Delete button in Saved runs removes a separately saved reviewer snapshot. Deleting one does not automatically delete the other. Neither operation deletes the global shortlist, AI thumbnail cache, target photograph, or Mapillary token.

Current Reproducibility Limitation

Repeated identical Large Area Searches can currently yield different numbers of unique images. The current discovery engine should be considered a POC and not a deterministic exhaustive crawl.

12. Saved Runs and Shortlist

12.1 Saved Runs

Control	What It Does
Run name	Optional name for the current reviewer state.
Save current run	Saves the current candidate set and its current AI score/timing information to SQLite.

Control	What It Does
Saved runs	Selects a previously saved reviewer run.
Load	Loads the selected saved run without repeating Mapillary discovery.
Delete	Deletes the selected saved reviewer run. This does not delete a Large Area Search job.
Export current CSV	Exports the current reviewer set and associated search/AI fields to CSV.

Persisting AI Scores

After an AI run completes, click Save current run if its AI scores, selected AI image type, and timing/performance statistics should persist with the saved run.

12.2 Shortlist

Control	What It Does
Shortlist count	Shows the number of images stored in the persistent shortlist.
Load shortlist	Makes the shortlist the current candidate set; useful before Thorough Multi-Crop.
Export shortlist CSV	Exports the shortlist and its stored metadata/AI fields.
Clear shortlist	Deletes all shortlist entries after confirmation.

The shortlist is stored in mapillary_runs.sqlite3 and persists across browser/server restarts.

13. AI Visual Similarity Ranking

13.1 AI Controls and Pull-Down Menus

Control	Options / Meaning
Comparison mode	Fast multi-crop for broad screening; Thorough multi-crop for deeper comparison.
AI image type	Both panorama and flat images; Panoramas only; Flat images only.
Mapillary thumbnail	1024 px recommended; 2048 px uses a higher-resolution source image.
Compute device	Auto; CPU; AMD ROCm GPU; NVIDIA CUDA GPU; DirectML GPU; Apple MPS.
Maximum images to score	0 means all images in the selected AI image-type subset; a positive value scores only the first N eligible images.
Keep downloaded Mapillary thumbnails	Retains thumbnails under ai_cache for much faster repeat runs.

13.2 AI Image-Type Filter

Version 2.4 can restrict AI ranking after discovery without deleting or replacing the underlying candidate set.

- Both panorama and flat images — scores every eligible candidate.
- Panoramas only — scores only candidates marked as panorama images.

- Flat images only — scores only non-panorama candidates.

The interface displays the current candidate counts for total, panorama, flat, and any unknown image-type records. If a Pano or Flat subset contains zero images, that option is disabled. The browser and server also prevent a zero-image AI job from starting.

After a filtered AI run, Sort by AI similarity places the images scored by that run first while preserving the remainder of the full candidate set.

13.3 AI Buttons

Button	What It Does
Check AI environment	Checks dependencies, available compute backends, hardware details, and current AI cache usage.
Rank current run with AI	Starts DINOv2 scoring for the currently selected AI image-type subset.
Cancel AI run	Requests cancellation; the current image/crop batch may finish first.
Sort by AI similarity	Orders scored images by similarity, places the most recently scored filtered subset first, and moves the viewer to item #1.
Restore distance order	Restores the original nearest-to-farthest ordering and moves the viewer to item #1.
Clear AI thumbnail cache	Deletes cached Mapillary thumbnails after confirmation; saved runs and scores remain.

13.4 AI Timing and Performance Status

Version 2.4 records and displays timing information for Fast and Thorough AI jobs. During a run, the status shows the start time and live elapsed duration. At completion it reports the start time, end time, elapsed duration, images scored, failures, cache hits, average images/second, and compute device.

AI ranking complete (Both): 30,794 scored, 4 failed, 30,794 cache hits
 Started 2026-09-27 08:14:37
 Ended 2026-09-27 08:38:51
 Elapsed 24m 14s
 Average rate 21.18 images/sec
 Device: AMD ROCm GPU — AMD Radeon(TM) 8060S Graphics

The latest AI job statistics are included when the current run is saved and are also included in the current-run CSV export.

13.5 AI Top-Match Table

Button	What It Does
Go	Moves the main candidate viewer directly to the selected AI-ranked image.
Add to shortlist	Adds the AI-ranked image to the persistent shortlist. Once added, the control indicates that it is shortlisted.

14. Target Photograph Controls

Control	What It Does
File chooser / Browse	Selects a JPEG, PNG, or WebP target image from the local computer.
Use selected target	Validates, uploads, and activates the selected target image.
Target image	Displays the active target; clicking it opens the image in a separate browser tab.
Target status	Reports upload state, filename, size, and validation errors.

Target requirements:

- Supported formats: JPEG, PNG, WebP.
- Maximum upload size: 30 MB.
- The browser performs a decode test before upload.
- The server verifies the image signature and, when Pillow is installed, verifies decodability.

A corrupt or unsupported target should be rejected with a clear error before an AI ranking job is started.

15. Candidate Viewer Controls

15.1 Viewer Pull-Down

Option	Behavior
MapillaryJS interactive	Displays the candidate in Mapillary's interactive viewer. The Mapillary token is available client-side for this mode.
2048 px thumbnail	Displays a still Mapillary thumbnail instead of the interactive viewer.

15.2 Navigation and Candidate Buttons

Control	What It Does
Previous	Moves to the previous candidate in the current ordering.
# field	Enter a candidate number; numbering begins at 1.
Go	Jumps to the candidate number entered in the # field.
Next	Moves to the next candidate.
Add to shortlist	Adds the current candidate to the persistent shortlist.
Remove from shortlist	Appears for an already-shortlisted candidate and removes it.
Open in Mapillary	Opens the current Mapillary image in a new browser tab.
Copy ID	Copies the current Mapillary image ID to the system clipboard.

16. Current Image Metadata

The metadata panel can display:

- Mapillary image ID
- Distance from the search center
- Latitude and longitude
- Capture date/time

- Compass angle
- Panorama yes/no
- Camera type
- Sequence ID
- Mapillary creator username when available
- AI similarity score
- AI best crop
- AI model
- AI comparison mode
- AI image type used for scoring

17. Recommended AI Workflows

17.1 Standard Search Workflow

1. Run Standard Search.
2. Confirm that the target is uploaded.
3. Choose the AI image type to score.
4. Select Fast Multi-Crop, 1024 px, and Auto compute device.
5. Keep the thumbnail cache enabled.
6. Run AI ranking.
7. Review the highest scores and add promising candidates to the shortlist.
8. Load the shortlist.
9. Run Thorough Multi-Crop on the shortlist.
10. Save the current run if the scores and timing should persist.

17.2 Large Area Search Workflow

1. Select Large Area Search and enter an overall radius greater than 1,500 m.
2. Start the Large Area Search and allow the cells to complete.
3. Review the scrollable cell list for failures or cap warnings.
4. Retry failed cells if appropriate.
5. Click Load combined results.
6. Choose Both, Panoramas only, or Flat images only for the AI run.
7. Run Fast Multi-Crop on the broad candidate set.
8. Build a shortlist from the top results.
9. Load the shortlist and run Thorough Multi-Crop.
10. Save/export the reviewer run as needed.

17.3 Why the Cache Matters

Large AI runs can spend substantial time downloading and preparing Mapillary thumbnails. Once the thumbnails are cached, repeat Fast and Thorough runs can be dramatically faster. Do not clear ai_cache unless disk space or troubleshooting requires it.

18. Hardware Acceleration

Backend	Intended Platform / Notes
CPU	Universal fallback. PyTorch uses the available CPU-thread configuration.
AMD ROCm	Preferred AMD path on a compatible ROCm-enabled PyTorch installation.
NVIDIA CUDA	For compatible NVIDIA GPUs with a CUDA-enabled PyTorch build.
Apple MPS	Potential Apple Silicon GPU path; v2.4 has not been tested on macOS.
DirectML	Optional Windows fallback when torch-directml is installed; ROCm is preferred on supported AMD systems.

Hardware check:

```
python .\check_ai_hardware.py
```

The application currently targets CPU/GPU acceleration. It does not offload DINOv2 to the Ryzen AI NPU.

19. Files, Database, Cache, and Backups

Location	Purpose
mapillary_runs.sqlite3	Saved runs, shortlist, and Large Area Search job/cell state.
target_uploads\	Uploaded target images and current-target tracking.
ai_cache\thumb_1024\	Optional cached 1024-pixel Mapillary thumbnails.
ai_cache\thumb_2048\	Optional cached 2048-pixel Mapillary thumbnails.
Hugging Face cache	Downloaded DINOv2 model files stored by Transformers outside the project folder.

Backup

Stop serve.py before making a file-level backup. At minimum, preserve mapillary_runs.sqlite3. Preserve config.json, target_uploads, and ai_cache if those should also be retained.

SQLite Journal Files

SQLite may create temporary or companion journal/WAL files while the application is running. Do not manually delete active SQLite companion files as a troubleshooting step unless the server is stopped and a backup exists.

20. CSV Exports

The current-run CSV includes geographic search fields, candidate metadata, AI score fields, the v2.4 AI image-type field, and the latest saved AI timing/performance statistics.

Typical fields include:

- Run name and saved time
- Mapillary image ID, latitude, longitude, and distance
- Capture time, compass angle, panorama flag, sequence ID, and camera type
- AI similarity, best crop, model, mode, AI image type, thumbnail size, and scoring time

- AI run start time, finish time, elapsed seconds, scored count, failed count, cache hits, compute device, and images/second
- Center latitude/longitude, radius, discovery image-type filter, date fields, maximum API images, and API images examined

21. Troubleshooting

Symptom	Suggested Action
sqlite3.OperationalError: disk I/O error at startup	If the application is installed in a deeply nested Windows path, move the entire folder to a short location such as D:\IGE_v2_4 and retry.
Mapillary token NOT configured	Set MAPILLARY_ACCESS_TOKEN or enter the token in config.json, then restart serve.py.
No Mapillary images found	Check coordinates, radius, and discovery image type. Try All imagery or a larger radius. Note that current discovery is not guaranteed exhaustive.
Repeated identical searches return different counts	Known v2.4 discovery limitation. Preserve results if needed and do not assume an identical rerun will return the same image-ID set.
Mapillary error: start_captured_at is missing the timezone	Known v2.4 date-filter bug. Leave Captured on/after and Captured on/before blank until the date handling is fixed.
Large Area Search radius $\leq 1,500$ m	Use Standard Search. Large Area Search accepts 1,501–50,000 m.
Large Area Search completed but reviewer still shows another set	Click Load combined results.
CAP REACHED	Increase Maximum API images, reduce the area, or divide the search. In Large Area Search the cap applies per cell.
AI dependencies missing	Run install_ai.ps1 or python -m pip install -r requirements-ai.txt, then restart serve.py.
AutoImageProcessor / Torchvision error	Confirm torchvision is installed in the same Python environment.
Rank current run with AI is disabled	A candidate set and uploaded target are required; AI must not already be running.
Panoramas only or Flat images only unavailable	The selected candidate set contains zero images of that type; choose another AI image type.
Corrupt target rejected	Choose another valid JPEG, PNG, or WebP image.
GPU option unavailable	The active PyTorch environment does not expose that backend. Use CPU or install the appropriate backend-specific PyTorch build.
AI run is slow	Use Fast mode, 1024 px, caching, GPU acceleration if available, or reduce the candidate set.
Port 8000 already in use	Start with python .\serve.py --port 8001.
MapillaryJS viewer problem	Switch Viewer to 2048 px thumbnail.
Disk usage grows	Clear AI thumbnail cache only if needed; old target uploads can also accumulate.

22. Security and Privacy Notes

Do Not Publish the Current POC Unchanged

The local application exposes the Mapillary token to the browser for MapillaryJS. A knowledgeable browser user may be able to inspect it. Use this build only in controlled local circumstances where that exposure is acceptable.

- Do not distribute a ZIP containing a real Mapillary token.
- Do not commit a configured config.json to a public repository.
- Uploaded target images are stored locally and are not encrypted by the application.
- The SQLite database is local and unencrypted.
- For a future shared web deployment, credentials should remain server-side and user data should be isolated.

23. Known Limitations and POC Boundaries

- Proof of concept; not production hardened.
- Tested on Windows; Linux and macOS have not been formally tested.
- Very long Windows application paths can cause SQLite disk I/O errors.
- Repeated identical Mapillary searches can return different candidate counts and image IDs.
- The current Mapillary discovery engine should not be treated as exhaustive or reproducible.
- Captured on/after and Captured on/before currently send timestamps without the timezone Mapillary requires and should be left blank.
- Large Area Search still requires a manual Load combined results step in v2.4.
- Large Area Search jobs and Saved runs are separate records with separate Delete controls.
- Single-user local architecture; no authentication or user authorization.
- Mapillary token is browser-visible when using MapillaryJS.
- Mapillary coverage can be sparse or absent.
- AI similarity is not a probability and does not prove location identity.
- Only one AI ranking job is intended to be active at a time.
- The app does not use an NPU.
- Target-upload history is not automatically purged.
- A production web deployment requires additional work for secrets, authentication, storage, request limits, and concurrency.

24. Quick Reference

Install / Start

```
# Recommended short install path
D:\IGE_v2_4

# Optional virtual environment
python -m venv .env
.\env\Scripts\Activate.ps1

# Mapillary token for this PowerShell session
$env:MAPILLARY_ACCESS_TOKEN="MLY|YOUR_TOKEN_HERE"
```

```
# Optional AI packages
python -m pip install -r .requirements-ai.txt

# Hardware check
python .\check_ai_hardware.py

# Start
python .\serve.py

# Browser
http://localhost:8000/
```

Recommended First-Pass AI Settings

Setting	Recommended Value
Comparison mode	Fast multi-crop
AI image type	Both, unless there is a reason to isolate panoramas or flat imagery
Mapillary thumbnail	1024 px
Compute device	Auto
Maximum images to score	0 (all eligible images)
Keep thumbnails	Checked

Recommended Second Pass

Load shortlist → Thorough multi-crop → select desired AI image type → Auto → 1024 or 2048 px → Rank current run with AI.

Stop

Ctrl+C

Version Information

Manual version: 2.0

Application covered: Image Geolocation Estimation App v2.4

Application status: Proof of Concept

Platform testing status: Windows tested; Linux and macOS not formally tested.

Known current issues documented: Mapillary discovery reproducibility, date-filter timezone bug, manual combined-result loading, and Windows long-path SQLite errors.